

Projet MasterR 1 IC – Sujet de TER

Titre du sujet : *MARIO¹ : Une plateforme pour expérimenter facilement l'apprentissage par renforcement multi-agent dans les jeux vidéo*

Nom de l'encadrant : Julien Soulé
Affiliation : LCIS – Université Grenoble Alpes & Télécom Paris – Institut Polytechnique de Paris
Email de l'encadrant : julien.soule@hotmail.fr
Téléphone de l'encadrant : (+33) 6 77 63 12 13
Année académique : 2025/2026
Nombre d'étudiants : 3

1. Présentation du sujet

Les systèmes multi-agents (Multi-Agent Systems – MAS) et l'apprentissage par renforcement multi-agent (Multi-Agent Reinforcement Learning – MARL) sont au cœur de nombreux travaux récents en intelligence artificielle, notamment pour la coordination d'agents dans des environnements collaboratifs ou compétitifs (jeux vidéo, robotique, systèmes de décision distribués, cyberdéfense/cybersécurité, etc.).

Malgré l'existence de nombreuses bibliothèques open-source (par exemple PettingZoo, PyTorch RL, RLlib), leur utilisation reste difficile pour des non-experts : choix et paramétrage des algorithmes, pré-configuration des environnements, gestion des hyper-paramètres, automatisation des phases d'entraînement et de test, etc.

Le projet **MARIO** a pour objectif de développer une plateforme expérimentale qui rende **l'utilisation du MARL plus accessible** pour des jeux vidéo multi-agents, en s'appuyant sur des bibliothèques existantes mais en automatisant autant que possible les tâches techniques habituellement réalisées à la main.

L'idée centrale est de **proposer un framework Python** qui fournisse une API simple d'utilisation, complétée par une interface en ligne de commande (CLI) et éventuellement une interface graphique (GUI), permettant de :

- sélectionner un ou plusieurs environnements de jeux collaboratifs/compétitifs,
- choisir un algorithme d'apprentissage par renforcement (MARL/RL) ou laisser un processus automatique de sélection et de réglage s'exécuter,
- lancer des expériences d'entraînement et de test,
- visualiser et analyser les résultats.

2. Objectifs du projet

Objectif général

Simplifier l'utilisation de techniques de MARL pour des jeux vidéo multi-agents en développant une plateforme générique, accessible via des outils existants.

1. MARIO = Multi-Agent Reinforcement learning Interface and Optimization

Objectifs détaillés

- Concevoir et implémenter un **framework Python** fournissant une API pour :
 - la sélection et l’initialisation d’environnements multi-agents (par ex. via Petting-Zoo) ;
 - la configuration d’algorithmes MARL/RL (ex. PPO, MAPPO) ;
 - la gestion intégrée des phases de pré-configuration, entraînement et test.
- Mettre en place un mécanisme d’**optimisation automatique des hyper-paramètres** (HPO), par exemple avec `Optuna`, avec :
 - des intervalles de valeurs par défaut suffisamment larges pour une utilisation générique ;
 - la possibilité pour l’utilisateur de préciser ses propres intervalles.
- Proposer une **interface CLI** permettant d’utiliser la plateforme sans écrire de code Python.
- En fonction du temps disponible, développer une **GUI web** (par exemple en Vue.js ou React) connectée à un backend Python (Flask/FastAPI).
- Démontrer le fonctionnement de la plateforme sur **au moins un jeu vidéo multi-agent** de référence (ex. Overcooked-AI), puis sur un ou deux environnements supplémentaires si possible.

3. Travail à réaliser par le groupe TER

Le travail attendu pourra être structuré en plusieurs phases.

3.1 Phase 1 – Revue de littérature et montée en compétences

- Prise en main des concepts de base du RL et du MARL (politiques, espace d’actions, d’observations, fonction de récompense, etc.).
- Étude des bibliothèques existantes :
 - PettingZoo pour les environnements multi-agents ;
 - tutoriels MARL de PyTorch RL (par ex. multi-agent PPO) ;
 - ressources théoriques sur le MARL (livre « Foundations and Modern Approaches to MARL »).
- Réalisation d’une **revue de littérature** sur les frameworks et plateformes existants, en se focalisant sur des critères comme : utilisabilité, généricité, automatisation, efficacité, visualisation.
- Construction d’un **tableau comparatif** : en lignes les travaux/outils, en colonnes les critères ; les cases indiquent quels critères sont couverts ou non (afin d’identifier les « gaps »).

3.2 Phase 2 – Conception de la plateforme MARIO

- Définition de l’architecture logicielle du framework (gestion des environnements, gestion des algorithmes, module HPO, module d’entraînement, module de test, interface utilisateur).

- Définition d’une **API Python** simple (par exemple une classe ou quelques fonctions principales) permettant d’utiliser la plateforme en quelques lignes de code.
- Spécification des points d’extension pour ajouter facilement de nouveaux environnements de jeux.

3.3 Phase 3 – Développement du framework Python

- Implémentation du chargement et de la gestion d’environnements multi-agents (par ex. via PettingZoo).
- Intégration d’un ou plusieurs algorithmes MARL (par ex. MAPPO) pour les jeux considérés.
- Mise en place d’un module d’**optimisation d’hyper-paramètres** utilisant **Optuna** : définition des espaces de recherche par défaut, prise en compte d’espaces fournis par l’utilisateur.
- Gestion de l’entraînement : lancement d’expériences, sauvegarde des modèles, logs d’apprentissage, etc.
- Gestion du test : évaluation des politiques obtenues, éventuellement sur des variantes de l’environnement (seeds différentes, configurations légèrement modifiées) pour évaluer la robustesse.

3.4 Phase 4 – Interfaces CLI et GUI (optionnelle)

- Développement d’une **interface en ligne de commande (CLI)** permettant d’appeler la plateforme sans écrire de code :

```
mario train --env overcooked --algo mappo --auto-hpo
mario test --model ./models/run42 --render
```

- (Optionnel) Développement d’une **interface web** (Vue.js / React) appelant un backend Python (Flask/FastAPI), permettant de : choisir un environnement, un algorithme, configurer les hyper-paramètres, lancer un entraînement et suivre l’apprentissage.

3.5 Phase 5 – Expérimentations et évaluation

- Mise en œuvre de la plateforme sur un premier jeu vidéo multi-agent de référence (par exemple Overcooked-AI).
- Ajout, si possible, d’un ou deux environnements supplémentaires pour montrer la généralité de la plateforme.
- Réalisation d’expériences pour évaluer :
 - la facilité de configuration (nombre de paramètres à renseigner, scripts nécessaires, etc.) ;
 - la performance des politiques apprises ;
 - la robustesse par rapport aux variations d’environnement.
- Analyse des résultats, discussion des limites et des perspectives.

4. Démonstrateur attendu

Le démonstrateur final devra comprendre :

- un **framework Python** installable (par exemple via un dépôt Git) avec une API documentée ;
- une **interface CLI** permettant d'utiliser MARIO sans écrire de code ;
- des **tutoriels simples** montrant comment entraîner et tester des agents sur un jeu vidéo multi-agent ;
- un mécanisme d'**optimisation automatique des hyper-paramètres** ;
- au moins un environnement de jeu (ex. Overcooked-AI), idéalement deux ou trois ;
- éventuellement une **interface web** de démonstration si le temps le permet.

5. Livrables

1. **Rapport 1** : cadrage du sujet et revue de littérature
 - contexte et problématique ;
 - revue de littérature et tableau comparatif des outils existants selon plusieurs critères (utilisabilité, généricité, automatisation, etc.) ;
 - identification des « gaps » et positionnement de MARIO ;
 - proposition d'architecture de la plateforme ;
 - organisation de l'équipe et planning (diagramme de Gantt).
2. **Code source** du framework MARIO (API Python, CLI, éventuellement GUI), versionné avec Git et correctement documenté.
3. **Rapport 2** : description détaillée de la contribution et évaluation expérimentale
 - architecture et choix d'implémentation ;
 - description des expériences, résultats et analyses ;
 - discussion critique et perspectives d'évolution.
4. **Soutenance finale** incluant une démonstration de la plateforme.

6. Compétences développées

- Compréhension des concepts de base en apprentissage par renforcement et en MARL.
- Conception d'architectures logicielles pour des plateformes d'IA.
- Programmation avancée en Python, intégration de bibliothèques existantes.
- Mise en place d'optimisation d'hyper-paramètres (HPO) avec des outils tels que **Optuna**.
- Réalisation d'expériences et analyse de résultats.
- Travail en équipe, gestion de projet (Scrum/Kanban), utilisation d'outils de versionnement (Git).

7. Technologies et outils

- **Langages** : Python (obligatoire), JavaScript/TypeScript (pour la GUI éventuelle).
- **Bibliothèques** :
 - PettingZoo (environnements multi-agents) ;
 - PyTorch RL ou bibliothèque équivalente pour les algorithmes MARL/RL ;
 - Optuna (optimisation d'hyper-paramètres) ;
 - Flask ou FastAPI (backend web) ;
 - Vue.js ou React (frontend web, optionnel).
- **Outils** : Git, environnement de développement Python (VSCode, PyCharm, ...), éventuellement Docker.

8. Positionnement du sujet

Niveau d'innovation : ☐ Très innovant ☒ Innovant ☐ Classique

Documentation
disponible : ☒ Beaucoup ☐ Moyenne ☐ Faible ☐ Aucune

Niveau d'abstraction : ☐ Théorique ☒ Théorique/Pratique ☐ Pratique

Quantité de
développement : ☒ Beaucoup ☐ Modérée ☐ Peu

Difficulté des
algorithmes : ☒ Élevée ☐ Modérée ☐ Simple

Interaction avec d'autres
composants logiciels : ☐ Écosystème très complexe ☒ Complexité intermédiaire ☐ Application isolée

9. Encadrement

- **Temps d'encadrement hebdomadaire** : environ une demi-journée par semaine (présentiel ou visio) + échanges asynchrones (mail, Git, etc.).
- **Contraintes particulières** : aucune contrainte forte, mais une bonne autonomie des étudiants est attendue, en particulier pour la partie développement.

10. Références

- Machine Learning / connexionniste : <https://www.youtube.com/watch?v=8d8Ko73XGeY&list=PLl1IO-qAzf2Sa006JR1RMtqpu6q6BWdcfR>
- PettingZoo : <https://pettingzoo.farama.org/>
- Tutoriel multi-agent PPO (PyTorch RL) : https://docs.pytorch.org/rl/stable/tutorials/multiagent_ppo.html
- Vidéo présentation MARL : <https://www.youtube.com/watch?v=QfYx5q0Q75M&t=1274s>
- Livre MARL : <https://www.mar1-book.com/download/mar1-book.pdf>
- Overcooked-AI : https://github.com/HumanCompatibleAI/overcooked_ai
- Optuna : <https://optuna.org/>